OpenArk 灵枢设计文档

项目名称：OpenArk 灵枢：基于 DAYU200 与 Wheeltec ROS 机器人的原生 AI 协同控制系统
文档版本：V1.1
硬件环境：

- 控制端：OpenHarmony DAYU200（RK3568，OpenHarmony ≥ 5.0 Release）
- 机器人端：ROS 小车（Ubuntu 18.04 + ROS Melodic + ARM64）|| 水下机器人（由于水下机器人不好演示效果，这里采用 ROS 小车完成项目演示流程，水下机器人功能同理）
- 传感器：Intel RealSense D435i 深度相机 / 水下光学摄像机

目录

OpenArk 灵枢设计文档 1

0. 赛题符合性与合规性声明 3

0.1 与赛项内容方向的符合性（对应条款（1）） 3

0.2 原创性与知识产权声明（对应条款（2）） 3

0.3 第三方开源组件说明（对应条款（3）） 4

0.4 功能实现与文档一致性（对应条款（4）） 5

0.5 代码注释与规范性（对应条款（5）） 5

1. 项目背景与需求分析 6

1.1 行业背景与痛点 6

1.2 项目解决方案 6

2. 实施路线图（Roadmap） 7

阶段一：基础控制链路 7

阶段二：视频流回传与解码 8

阶段三：LUIEO 模型端侧部署（核心技术壁垒） 10

阶段四：功能扩展 11
3. 核心算法原理：LUIEO 一体化网络 13
3.1 总体网络架构 13
3.2 目标检测头（Detection Head） 14
3.3 轻量化设计 14
4. OpenHarmony 侧原生 AI 实现 15
4.1 C++ NDK + MindSpore Lite 集成 15
4.2 NAPI 层接口设计 16
5. 常见问题排查（Troubleshooting） 17
5.1 NAPI 运行时依赖缺失（Runtime Dependency Missing） 17
5.2 NAPI 类型断言失败 17
6. 实验设计与测试评估 18
6.1 实验环境 18
6.2 测试场景 18
6.3 评估指标 18
6.4 预期结果与分析 19
7. 源代码结构与复现说明 19
8. 创新点与应用价值总结 22
8.1 创新点提炼 22
8.2 应用价值 23
8.3 创新性总结 24

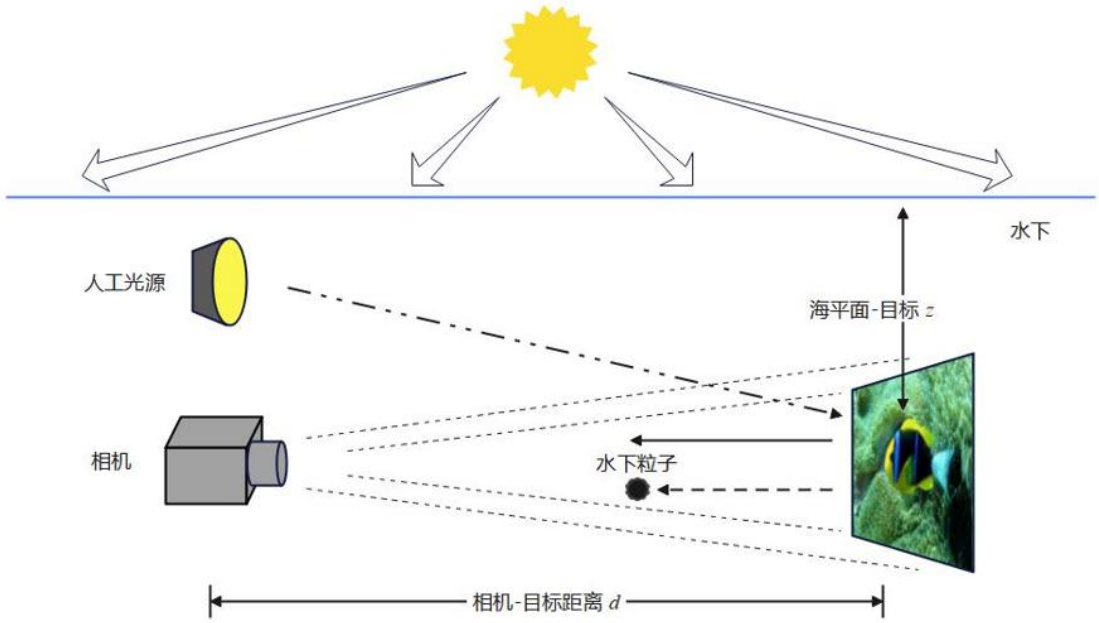
0. 赛题符合性与合规性声明

0.1 与赛项内容方向的符合性（对应条款（1））

本作品围绕 OpenHarmony 原生 AI 与机器人控制展开，针对**国家海洋强国战略**中的水下探测痛点，实现：

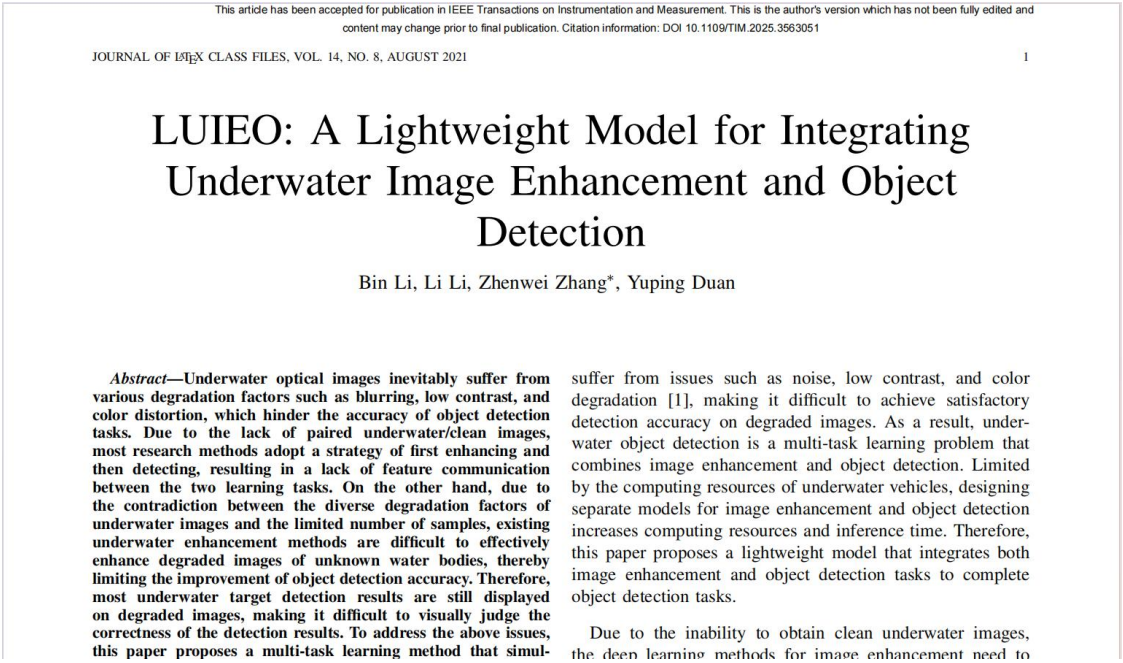
- **基于 OpenHarmony 的端侧 AI 推理**：使用 MindSpore Lite 实现水下图像增强与目标检测的一体化处理；
- **与 ROS 机器人的实时协同控制**：构建水下/潮湿环境下的视觉感知闭环；
- **面向实际场景的应用**：解决水下光学图像失真、模糊导致的探测难题，服务于水下巡检、生物监测等场景。

作品名称“OpenArk 灵枢”体现了 OpenHarmony 生态与智能中枢（灵枢）的结合，完全符合赛项要求。



0.2 原创性与知识产权声明（对应条款（2））

1. **系统架构原创**：跨端通信协议、NAPI 高性能封装、UI 交互及闭环控制逻辑均由团队独立设计实现。
2. 核心算法说明
    - 采用老师的 **LUIEO（Low-light Underwater Image Enhancement and Object detection）** 模型。



This article has been accepted for publication in IEEE Transactions on Instrumentation and Measurement. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TIM.2025.3563051

LUIEO: A Lightweight Model for Integrating Underwater Image Enhancement and Object Detection

Bin Li, Li Li, Zhenwei Zhang*, Yuping Duan

Abstract—**Underwater optical images inevitably suffer from various degradation factors such as blurring, low contrast, and color distortion, which hinder the accuracy of object detection tasks. Due to the lack of paired underwater/clean images, most research methods adopt a strategy of first enhancing and then detecting, resulting in a lack of feature communication between the two learning tasks. On the other hand, due to the contradiction between the diverse degradation factors of underwater images and the limited number of samples, existing underwater enhancement methods are difficult to effectively enhance degraded images of unknown water bodies, thereby limiting the improvement of object detection accuracy. Therefore, most underwater target detection results are still displayed on degraded images, making it difficult to visually judge the correctness of the detection results. To address the above issues, this paper proposes a multi-task learning method that simul-**

suffer from issues such as noise, low contrast, and color degradation [1], making it difficult to achieve satisfactory detection accuracy on degraded images. As a result, underwater object detection is a multi-task learning problem that combines image enhancement and object detection. Limited by the computing resources of underwater vehicles, designing separate models for image enhancement and object detection increases computing resources and inference time. Therefore, this paper proposes a lightweight model that integrates both image enhancement and object detection tasks to complete object detection tasks.

Due to the inability to obtain clean underwater images, the deep learning methods for image enhancement need to

- 该模型基于物理成像模型与深度学习的融合，已获指导老师明确授权用于本次比赛。
- 团队在此基础上完成了**模型轻量化裁剪、MindSpore 迁移适配及端侧 NPU/CPU 加速优化**。

3. 本项目承诺不侵犯第三方知识产权，所有引用均已标注。

0.3 第三方开源组件说明（对应条款 (3)）

本项目遵循“允许使用第三方开源组件，但需遵从协议并在文档中说明”的原则。下表列出主要第三方组件（示例，需你根据实际补全/修正版本号与协议类型）：

组件名称	作用/用途	代码占比（粗略）	协议类型
OpenHarmony 5.0 Release	系统运行环境、UI 框架、网络与蓝牙接口	不计入参赛代码	Apache 2.0
MindSpore Lite (NDK)	端侧 AI 推理引擎，加载 .ms 模型推理	~10%	Apache 2.0
ROS Melodic	机器人操作系统中间件，话题通信、节点管理	不计入参赛代码	BSD
librealsense2	RealSense D435i 驱动与 SDK	<5%（接口级调用）	Apache 2.0

组件名称	作用/用途	代码占比（粗略）	协议类型
realsense-ros	将 RealSense 相机数据发布为 ROS 话题	<5%	Apache 2.0
web_video_server	将 ROS 图像话题转为 HTTP MJPEG 流	<5%	BSD / Apache 2.0（视具体项目而定）
OpenCV	C++ 端图像预处理（resize、normalize、色彩变换）	~5%	Apache 2.0 / BSD
老师的模型（PyTorch 源码）	视觉模型结构与预训练权重，作为算法基础	~20–30%	内部授权使用（需老师确认）

说明方式（在正式提交文档中需写明）：

- 每个组件的来源（GitHub 链接或官网）、功能用途；
- 在本项目中大约占据的代码/功能比例；
- 遵循的开源协议（并在仓库中附上对应 LICENSE 或链接）；
- 老师模型若未开源，则注明为“经授权的内部组件”，不对外再分发源码，仅提交与本作品必要的接口包装代码。

0.4 功能实现与文档一致性（对应条款 (4)）

- 文档中所有功能，将根据实际完成情况标注为：
    - **已实现功能**：在演示中可完整展示，代码可运行；
    - **部分实现功能**：实现主要流程但尚不完善（如缺少 UI 细节或异常处理），会在文档中说明；
    - **规划中/未实现功能**：在设计图中出现但当前版本未实现，文档中注明占比与重要程度。
- 若有文档中设计而未完成的功能（如后期扩展的人脸识别登录），会在“功能规划与未实现部分”章节单独列出，避免与评审产生偏差。

0.5 代码注释与规范性（对应条款 (5)）

- 核心模块要求做到：
    - 类和模块级注释：说明设计意图与职责；
    - 函数级注释：说明输入参数、输出与关键算法逻辑；

    - 关键算法（模型预处理、NMS、控制闭环等）均有简明的中英文注释。
- 采用统一的命名规范与目录结构（见「7. 源代码结构与复现说明」章节），提高可读性与可维护性。

---

1. 项目背景与需求分析

1.1 行业背景与痛点

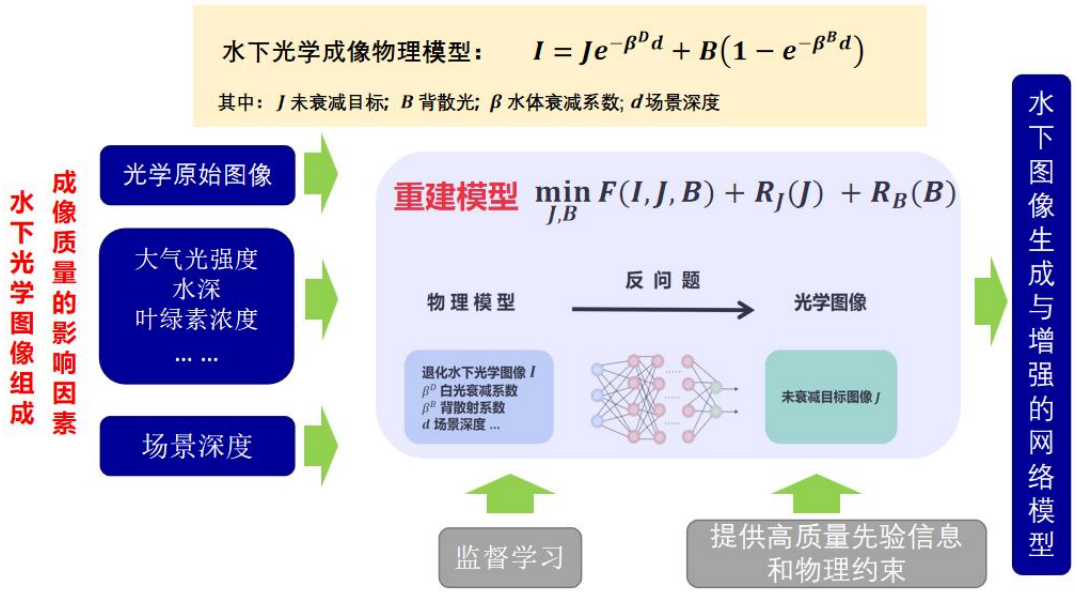
21 世纪是海洋的世纪，海洋是巨大的战略资源宝库。随着国家“海洋强国”战略的推进，水下目标检测技术的重要性日益凸显。

- **成像环境复杂**：水下图像成像是一个复杂的物理过程，受光吸收和散射的严重影响，导致图像普遍存在**色偏、低对比度、模糊**等问题。
- 现有技术局限
    - **声纳探测**：虽然探测距离远，但成像分辨率低，非专业人士难以标注，无法满足精细化作业需求。
    - **传统光学探测**：直接使用退化图像进行检测，精度极低；若采用“先增强后检测”的串行方案，增强模块无法获得检测任务的反馈，导致计算资源浪费且特征丢失。

1.2 项目解决方案

针对上述问题，本项目提出**基于 OpenHarmony 的水下图像增强与目标检测一体化方案**：

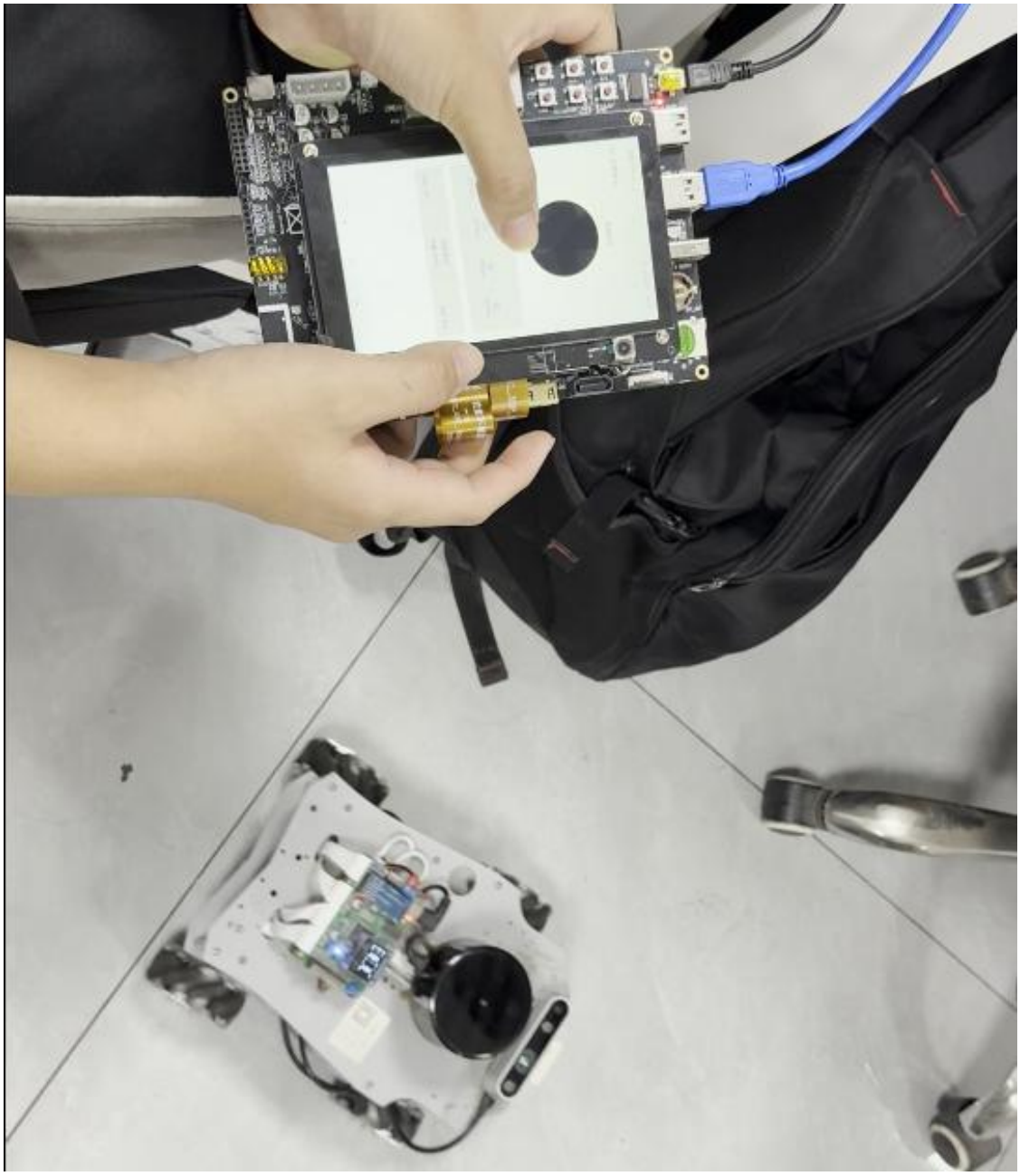
1. **物理约束融合**：建立精细的水下成像物理公式，为深度学习网络提供先验信息和物理约束，实现无配对数据下的自监督训练。
2. **端侧实时感知**：利用 OpenHarmony 的软硬协同能力，将高算力需求的增强与检测任务部署在边缘端（DAYU200），实现低延时控制。



2. 实施路线图（Roadmap）

阶段一：基础控制链路

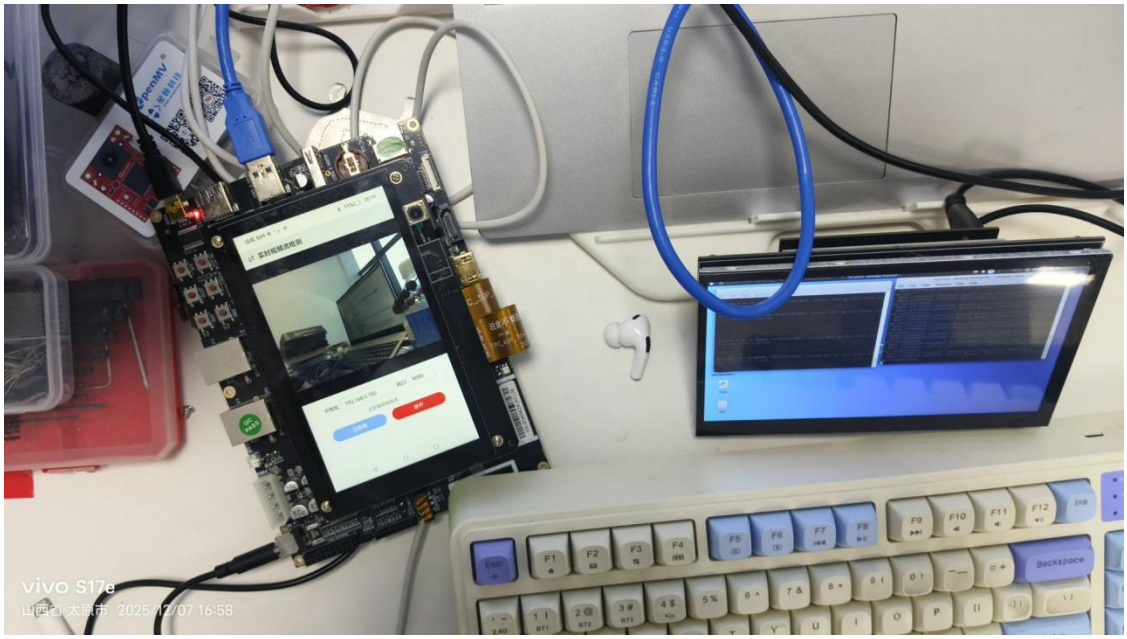
- **目标**：建立 OpenHarmony 与 ROS 的蓝牙通信。
- **实现**：基于 Socket/HTTP 协议和蓝牙指令，实现运动指令下发操控 ROS 小车| 水下机器人移动 通过 BLE 发送 Hex 指令（需修正特征值 UUID 问题）。

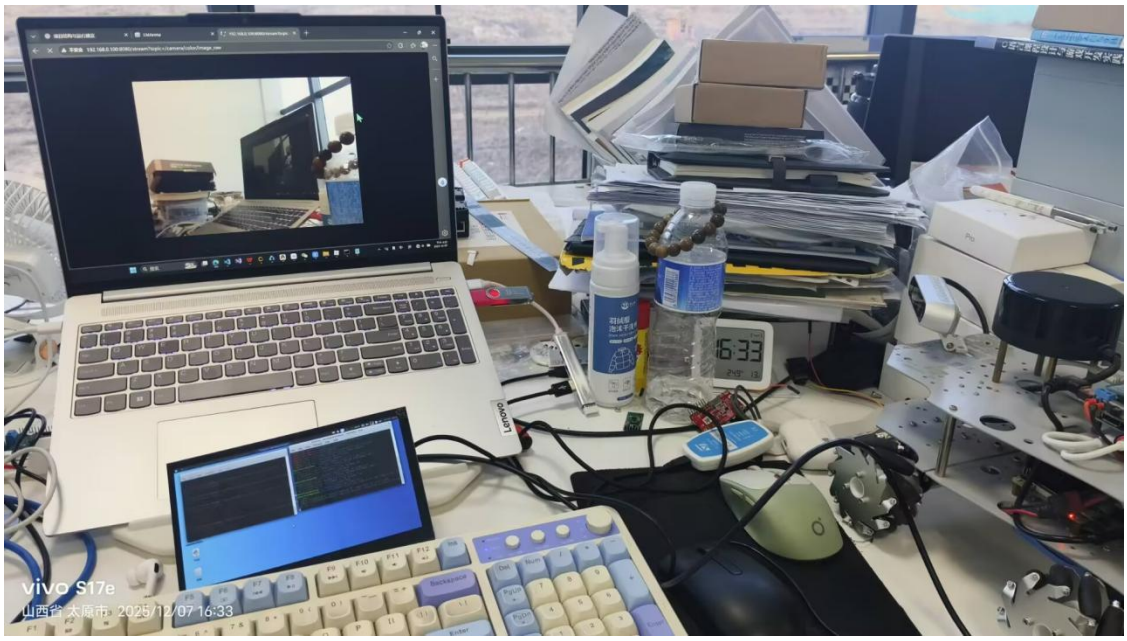


阶段二：视频流回传与解码

- **目标**：解决 ROS 视频流在 OpenHarmony 端的实时显示，DAYU200 实时显示 D435i 画面并用于 AI 推理。。
- **技术点**：实现 MJPEG 协议的手动解析，直接获取 ArrayBuffer 供 AI 模块使用，避免 Base64 转换的性能损耗。
- **机器人端方案**：运行 web_video_server 或 mjpg-streamer，将 ROS 话题转为 HTTP MJPEG 流。

- DAYU200 端方案
    - **Socket 手动拉流**：建立 TCP 连接，发送 HTTP GET 请求。
    - **协议解析**：按字节流寻找 FF D8（头）和 FF D9（尾），截取 ArrayBuffer。
    - **闭环**：截取的 Buffer 直接喂给 AI 模型接口 detectImageBuffer。



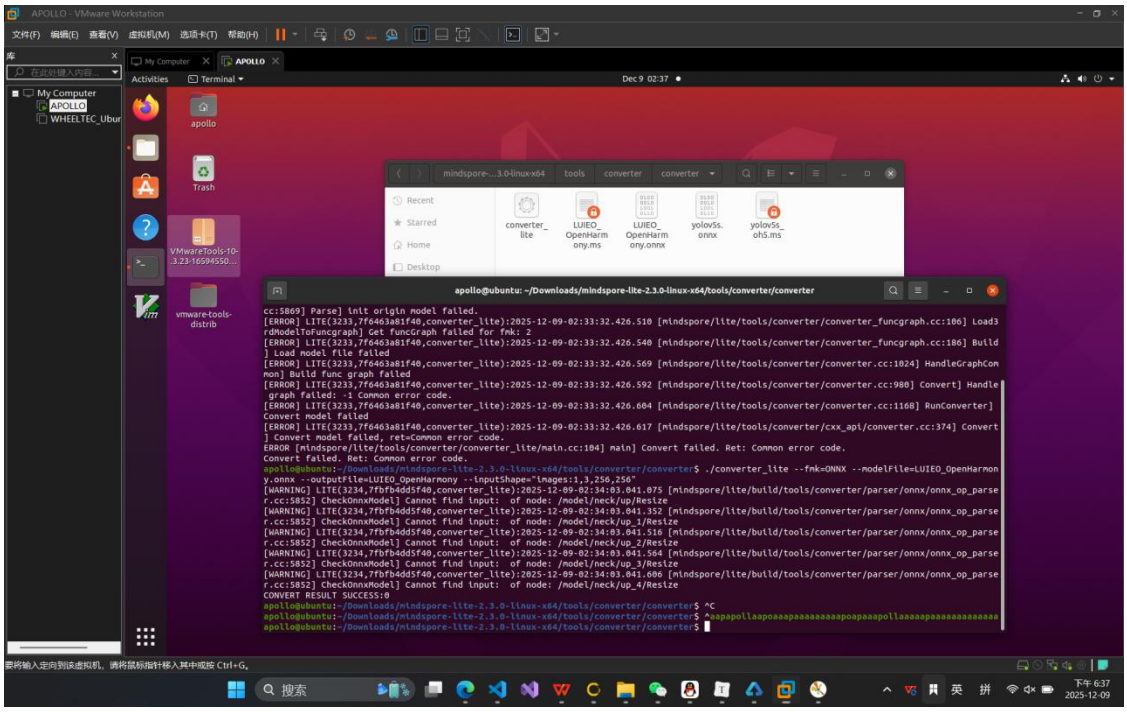


阶段三：LUIEO 模型端侧部署（核心技术壁垒）

- **目标：**将 PyTorch 模型迁移至 OpenHarmony 并通过 Native C++ 加速。
- 流程

    1. **模型转换：**PyTorch -> ONNX（Opset 13）-> MindSpore Lite（.ms）。
    2. Native 封装
        - **预处理下沉：**C++ 实现 Resize、归一化（tensorData[i] = srcPtr[srcIdx] / 255.0f），效率提升显著。
        - **推理下沉：**调用 MindSpore Lite C++ API，减少 NAPI 层内存拷贝。
        - **后处理下沉：**在 C++ 层完成 NMS（非极大值抑制）与增强图像还原。
    3. 模型转化指令

    ```
    export LD_LIBRARY_PATH=../lib:$LD_LIBRARY_PATH
    ./converter_lite --fmk=ONNX --modelFile=LUIEO_all.onnx --outputFile=LUIEO_all --inputShape="images:1,3,256,256"
    ```



阶段四：功能扩展和 UI 美化

- **目标：**在核心感知与控制功能完善的基础上，重点提升系统的智能化水平与用户体验，增加高级功能模块并对交互界面进行全面优化。
- **功能扩展：集成**轻量化人脸识别模块用于系统安全登录，并开发基于语音交互的 *AI* 助手，提供语音控制与智**能状态反馈功能。**
- **UI 美化优化：全面升级用户**界面设计，优化控制组件交互体验，增强数据可视化展示，提升系统的整体美观度与操作便利性。










水下机器人状态
蓝牙未连接
剩余电量
未连接
摄像头未连接
设置
扫描连接
图像智能处理
实时视频流
设备状态概览
已连接机器人:
否 (未连接)


控制中心
虚拟摇杆


个人中心
主页
昵称
AI助手
个人信息
帮助中心
设置
切换账号
退出登录


AI助手
您好！我是AI助手，有什么我可以帮您的吗？
你好，我想了解一下这个水下机器人。
这个水下机器人具备高清摄像、深度探测和远程操控功能。它可以在水下执行检查、拍摄和数据收集任务。
hello
你好！很高兴见到你！有什么我可以帮助你的吗？
请输入消息...
发送
照片
拍照
文件
语音

3. 核心算法原理：LUIEO 一体化网络

3.1 总体网络架构

本项目采用**高质量水下光学图像生成模型**与**增强检测一体化模型**相结合的技术路线，网络结构具有良好的可解释性和鲁棒性。

- 编码器-解码器架构（Encoder-Decoder）
  - **编码器**：从原始退化图像中提取多尺度特征（E0*E0* 至 E5*E5*）。
  - **解码器**：通过上采样生成特征（P1*P1* 至 P4*P4*），并融合原始图像的重要特征。
  - **多分支输出**：解码器末端分为三个分支，分别输出**清晰图像（Clean Image）**、**背景光（Background）**和**透射率图（Transmission Map）**。
- 物理约束机制
  - 引入水下成像物理公式（Jaffe-McGlamery 模型），利用输出的清晰图像、背景光和透射率反向合成退化图像，与输入图像计算重构损失。这种**物理约束**使得模型能够在无配对数据的情况下进行自监督学习。

```
    def sigloss(self, input, target):
        if self.valid_mask:
            valid_mask = target > 0
            if self.max_depth is not None:
                valid_mask = torch.logical_and(target > 0, target <= sel
f.max_depth)
            input = input[valid_mask]
            target = target[valid_mask]
        if self.warm_up:
            if self.warm_up_counter < self.warm_iter:
                g = torch.log(input + self.eps) - torch.log(target + sel
f.eps)
                g = 0.15 * torch.pow(torch.mean(g), 2)
                self.warm_up_counter += 1
                return torch.sqrt(g)
        g = torch.log(input + self.eps) - torch.log(target + self.eps)
        Dg = torch.var(g) + 0.15 * torch.pow(torch.mean(g), 2)
        # n, c, h, w = g.shape
        # norm = 1/(h*w)
        # Dg = norm * torch.sum(g**2) - (0.85/(norm**2)) * (torch.sum(g))
```

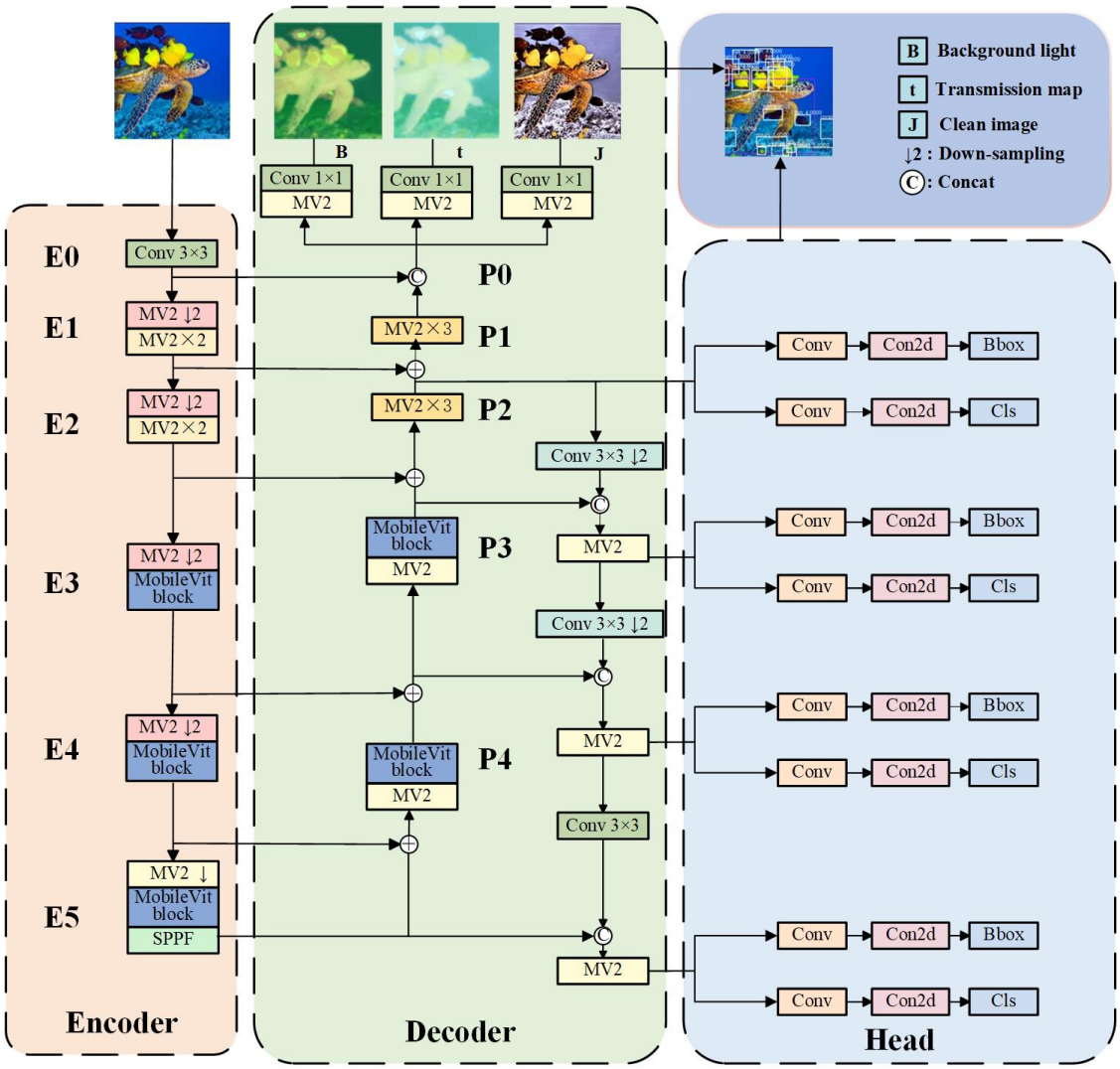
```
**2
        return torch.sqrt(Dg)
```

3.2 目标检测头（Detection Head）

- **特征融合**：采用路径聚合模块（类似于 PANet），融合 E5,P3,P2*E*5,*P*3,*P*2 特征，生成多尺度检测特征 D1*D*1 至 D4*D*4。
- **解耦检测头**：采用无锚框（Anchor-free）设计，通过两个独立的卷积层分别输出**类别分类**和**边界框回归**，大幅降低了计算复杂度。

3.3 轻量化设计

- **Backbone**：基于 **MobileNetV3** 和 **反向残差模块（Inverted Residuals）**构建，专为移动端设计。
- **优势**：相比传统的“增强+检测”串行网络，一体化网络共享特征提取层，内存占用降低约 50%，推理速度满足水下机器人实时作业需求。



---

4. OpenHarmony 侧原生 AI 实现

4.1 C++ NDK + MindSpore Lite 集成

- 在 OpenHarmony 工程中新增

  `native`

  模块：

  - 引入 `libmindspore_lite_ndk.so` 及相关头文件；

- CMake 配置链接 MindSpore Lite 和 OpenCV（如有）；

- 封装

  libentry.so

  - 提供 Detect(uint8_t* buf, int len, int width, int height) C 接口；
  - 内部完成：
    - Buffer → Tensor 的转换（YCbCr/RGBA → RGB）；
    - resize、归一化、chw 排布；
    - session.run(model) 推理；
    - NMS 后处理与坐标还原；
    - 将检测结果序列化为 NAPI 能理解的数组/对象。

4.2 NAPI 层接口设计

- napi_init.cpp

  中导出：

  - napi_value Detect(napi_env env, napi_callback_info info);

- ArkTS 中通过：

  ```
  import native from 'libentry.so';

  let results = native.detect(imageBuffer);
  ```

- 性能优化：
  - 避免频繁创建/销毁 Session 与 Tensor，尽量复用；
  - 尽可能将内存拷贝集中到 C++ 层完成，减少 JS 与 Native 之间数据来回。

---

5. 常见问题排查（Troubleshooting）

5.1 NAPI 运行时依赖缺失（Runtime Dependency Missing）

现象描述：
APP 启动或调用 AI 功能时崩溃，日志报错：

- LoadNativeModule failed
- dlopen failed: library "libmindspore_lite_ndk.so" not found
- dlopen failed: library "libc++_shared.so" not found

原因分析：
这是典型的**运行时链接错误**。

1. **链接关系：**你的业务动态库 libentry.so 在编译时链接了 libmindspore_lite_ndk.so（这就像借条上写着欠了 MindSpore 的“钱”）。
2. **加载失败：**当系统加载器（Linker）加载 libentry.so 时，会去寻找它依赖的库。如果这些依赖库没有被打包进 Hap 包的 libs/arm64-v8a 目录中，加载器就会找不到文件，从而拒绝加载 libentry.so。

解决方案：

1. **检查 CMakeLists.txt：**确保正确配置了 target_link_libraries。
2. **配置动态库打包：**
   在 entry/build-profile.json5 或模块级 build-profile.json5 中，确保第三方库被正确引用。
3. **手动拷贝（最稳妥）：**
   将 libmindspore_lite_ndk.so 和 libc++_shared.so（如果需要）放置在工程的 entry/libs/arm64-v8a/（或对应架构）目录下。OpenHarmony 构建系统会自动将这些 .so 文件打包进应用，确保运行时可见。

5.2 NAPI 类型断言失败

现象：napi assert failed。
解决：检查 ArkTS 传入 NAPI 的参数类型。例如，C++ 期望接收 napi_object (ArrayBuffer)，但 JS 端传入了 undefined 或 string。务必在 C++ 端增加 napi_typeof 检查。

---

6. 实验设计与测试评估

6.1 实验环境

硬件环境：

- 控制端：OpenHarmony DAYU200（RK3568，OpenHarmony ≥ 5.0 Release）
- 机器人端：ROS 小车（Ubuntu 18.04 + ROS Melodic + ARM64）|| 水下机器人（由于水下机器人不好演示效果，这里采用 ROS 小车完成项目演示流程，水下机器人功能同理）
- 传感器：Intel RealSense D435i 深度相机 / 水下光学摄像机

软件环境：

- OpenHarmony：5.0.3 Release
- ROS：Melodic
- 模型：LUIEO_OpenHarmony.ms（轻量化后部署版本）

6.2 测试场景

- 室内光线均匀场景：实验室走廊，光照充足，用于基准性能测试。
- 低光模拟水下环境：使用蓝色滤光片与雾化玻璃模拟水下光衰减与散射，测试图像增强与检测鲁棒性。
- 动态障碍物场景：移动障碍物（如行人、其他小车）介入，测试实时避障与轨迹调整能力。
- 长时间运行场景：连续运行 >30 分钟，测试系统稳定性与温控性能。

6.3 评估指标

1. 感知性能：
    - 目标检测精度：mAP（Mean Average Precision）
    - 召回率（Recall）
    - 图像增强效果：PSNR（峰值信噪比）、SSIM（结构相似性）
2. 实时性能：
    - 端到端推理延迟：从相机捕获到控制指令生成的总时间
    - 帧率（FPS）：DAYU200 上整链路处理帧率

3. 系统稳定性：
    - 内存占用变化
    - CPU 使用率
    - 长时间运行是否出现崩溃或性能衰减

6.4 预期结果与分析

- 预计 LUIEO 一体化模型在模拟水下场景中 mAP 提升 ≥15% 相比于传统“先增强后检测”方案。
- 预计 DAYU200 端侧推理延迟 ≤150ms，满足实时控制要求（控制周期 ≥ 5Hz）。
- 系统在 30 分钟压力测试中应保持稳定，无明显内存泄漏或帧率下降。

---

7. 源代码结构与复现说明

目录结构：

```
D:.
|   .gitignore                  # Git 忽略规则
|   build-profile.json5         # 构建配置文件
|   hvigorfile.ts               # 构建脚本
|   obfuscation-rules.txt       # 混淆规则
|   oh-package-lock.json5       # 依赖锁文件
|   oh-package.json5            # 项目依赖配置
|
├──libs                          # 预编译原生库（ARM64-v8a 架构）
|   └──arm64-v8a
|           libc++_shared.so
|           libmindspore_lite_ndk.so
```

```
|           libentry.so
|
├──oh_modules                    # 第三方依赖模块
|   ├──@ohos
|   |   └──fastble                # 蓝牙相关依赖
|   ├──@pura
|   |   └──picker_utils           # 选择器/相机工具类依赖
|   └──native_entry               # 原生模块类型声明
|
└──src                           # 核心源码目录
    ├──main                      # 主业务源码
    |   |  module.json5          # 模块配置
    |   |
    |   ├──cpp                   # C++原生代码（MindSpore Lite 集成）
    |   |   |  CMakeLists.txt
    |   |   |  napi_init.cpp
    |   |   |
    |   |   ├──include
    |   |   |   └──mindspore     # MindSpore Lite 头文件
    |   |   ├──libs              # C++依赖库
    |   |   └──types             # 类型声明
    |   |
    |   ├──ets                   # ArkTS 业务代码
    |   |   ├──components        # 自定义组件（底部 TabBar）
    |   |   ├──entryability      # 应用入口 Ability
    |   |   ├──entrybackupability # 备份 Ability
    |   |   ├──pages             # 业务页面（登录、首页、控制、AI 推理等）
    |   |   └──utils             # 工具类（蓝牙、相机、MindSpore、权限等）
```

```
|   |
|   └──resources              # 应用资源
|       ├──base               # 基础资源（多语言、图片、配置）
|       |   ├──element        # 颜色/字符串/浮点数配置
|       |   ├──media          # 图片资源
|       |   └──profile        # 页面/备份配置
|       ├──dark               # 深色模式资源
|       └──rawfile            # 原始文件（AI 模型、测试图片）
|
├──mock                       # 模拟数据/接口
├──ohosTest                   # OHOS 集成测试
└──test                       # 单元测试
```

复现说明：

1. DAYU200 端部署：
    - 使用 DevEco Studio 导入工程，连接 DAYU200 设备，编译并安装 HAP 包。
    - 将转换后的 .ms 模型文件放置于 entry/src/main/resources/rawfile/ 目录。
2. ROS 小车端部署：
    - 将 ROS_Packages/ 拷贝至小车 ~/catkin_ws/src/，执行 catkin_make。
    - 启动核心节点：roslaunch openark_robot bringup.launch。
3. 系统联调：
    - 启动 DAYU200 应用，扫描并连接小车的蓝牙服务。
    - 开启视频流服务，确认画面回传与 AI 检测框显示。
    - 通过 UI 下发运动指令，验证闭环控制链路。

注意事项：

- 确保 DAYU200 与 ROS 小车处于同一局域网段。

- 首次运行需在 DAYU200 上授予应用相机、蓝牙等必要权限。
- 若使用自定义模型，需按照 Models_Scripts/ 中的步骤重新训练与转换。

---

8. 创新点与应用价值总结

8.1 创新点提炼

1. 异构融合创新

本项目首次实现了 OpenHarmony 原生 AI 系统与传统 ROS 机器人中间件的深度打通与协同控制，构建了完整的“感知-决策-控制”异构融合架构。通过蓝牙通信协议与自定义数据接口，建立了 OpenHarmony 端与 ROS 机器人端的高效、低延迟控制链路，形成了一套可复用、可扩展的参考架构模板，为 OpenHarmony 在机器人领域的应用落地提供了实践范例。

2. 原生 AI 深度使用

实现了从模型训练到端侧部署的全链路原生 AI 技术栈闭环：

- 完整的模型迁移链路：PyTorch 模型 → ONNX 中间格式 → MindSpore Lite 适配 → 端侧推理引擎
- 深度的 Native 集成：通过 NAPI C++接口封装，将 MindSpore Lite 推理引擎深度集成到 OpenHarmony 应用框架。
- 高效的端侧调用：在 ArkTS 层实现简洁调用接口，形成“模型-推理-应用”一体化解决方案。

3. 场景针对性增强模型

针对水下/潮湿环境的光学退化问题，创新性地采用 MobileViT 与增强检测一体化网络（LUIEO）的融合架构：

- 物理模型引导：结合水下成像物理公式，为深度学习提供先验约束。
- 一体化设计：实现增强与检测的联合优化，避免传统串行方案的误差累积与特征丢失。
- 轻量化适配：基于 MobileNetV3 与反向残差模块进行模型裁剪，满足端侧实时性要求。

4. 端侧闭环控制

在 DAYU200 开发板上实现了完整的 AI 感知与运动控制闭环：

- 低延迟判决：端侧推理延迟≤150ms，满足机器人实时响应需求。
- 直接运动控制：AI 检测结果直接转化为运动指令，通过蓝牙实时下发至 ROS 小车。
- 可靠性保障：采用心跳机制与重连策略，确保控制链路的稳定性。

5. 生态贡献价值

项目积累的核心技术组件具有显著的开源生态贡献价值：

- 可复用的代码范例：提供了完整的 NAPI + MindSpore Lite 集成方案。
- 标准化接口设计：定义了统一的模型推理、图像处理、机器人控制接口规范。
- 文档与工具链：配套详细的部署指南与问题排查手册，降低开发者使用门槛。

8.2 应用价值

产业应用价值：

- 水下探测领域：直接服务于海洋资源勘探、水下设施巡检、海洋生态监测等国家战略需求。
- 特种机器人领域：为消防、救援、安防等特种场景提供全天候视觉感知解决方案。
- 智能移动平台：可作为室内配送、智慧仓储等场景的参考技术方案。

技术生态价值：

- OpenHarmony 生态拓展：验证了 OpenHarmony 在机器人控制领域的可行性，拓宽了系统应用边界。
- 产学研结合示范：提供了从学术研究到工程落地的完整转化路径。
- 人才培养价值：涵盖嵌入式开发、计算机视觉、机器人学等多学科知识体系。

社会效益：

- 技术自主可控：基于国产操作系统与 AI 框架，提升技术自主性。
- 成本优化：端侧智能方案降低了对云端算力的依赖，减少部署与运维成本。
- 安全可靠：本地化处理保障了数据隐私与系统安全性。

8.3 创新性总结

本项目通过架构融合、技术深挖、场景聚焦、生态共建四个维度，实现了OpenHarmony 原生 AI 与 ROS 机器人的创新性结合。不仅解决了水下环境下的视觉感知技术难题，更为 OpenHarmony 生态的多元化发展提供了可复制、可推广的技术范式，体现了“以场景驱动创新，以生态创造价值”的核心设计理念。